

Spring Framework Notes

Day 29 — Annotations Day 11 : Java Configuration & @ComponentScan

Making a Class Eligible for the IoC Container

This class has no source code of its own here — it is a placeholder note. If we are working on some functionality, we need one class dedicated to that particular operation.

Functionality of this class:

- Manage dependencies
- Share the object with the user

So, how many ways do we know to make a class eligible for the IoC container?

- XML Configuration — used when we have the source code or no source code, but it has drawbacks.
- Annotation + XML — used when we have the source code.
- Annotation — used when we have the source code.
- Java Configuration — used when we don't have the source code.

Example — Annotation + XML approach:

In this case, we know that the IoC container creates the object of the properties class.

Java Configuration approach:

This is used along with @Component, and when we declare the class using @Bean.

So, suppose we don't have the source code for a class — someone else has created this class. If we have to work on the functionality of this class, and we want to use the Spring Framework to manage its dependencies, we have to go for the Java Configuration approach.

Case 1 — When We Don't Have the Source Code (Motor Class)

Motor.java

```
public class Motor { // manage the object, some dependencies
    private String companyName;
    private int capacity;

    public void setCompanyName(String companyName) {
        this.companyName = companyName;
    }
}
```

```

    public void setCapacity(int capacity) {
        this.capacity = capacity;
    }

    public void start() {
        System.out.println("Motor started!!");
    }

    @Override
    public String toString() {
        return "Motor [companyName=" + companyName + ", capacity=" + capacity +
        "]\n";
    }
}

```

JavaConfig.java

```

package com.day29;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;
import org.springframework.core.env.Environment;

@Configuration
@PropertySource("classpath:com/day29/application.properties")
public class JavaConfig {

    @Autowired
    Environment environment;

    @Bean
    public Motor motor() {
        Motor motor = new Motor();

        motor.setCompanyName(environment.getProperty("companyName"));
        motor.setCapacity(
            Integer.parseInt(environment.getProperty("capacity")));
        return motor;
    }
}

```

Main.java

```

package com.day29;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

```

```

        ApplicationContext context = new
            AnnotationConfigApplicationContext(JavaConfig.class);

        Motor bean = context.getBean("motor", Motor.class);
        System.out.println(bean);
    }
}

```

We will get the exact output.

Case 2 — When We Have the Source Code (Vehicle Class)

Now, suppose we have a class for which we have the source code — the Vehicle class. Inside the Vehicle class, we are using the functionality of the Motor class. Since we have the source code, we can make this class a bean definition and manage the dependency injection ourselves in this class by using the Annotation-based approach.

Vehicle.java

```

@Component
public class Vehicle {

    @Value("${brandName}")
    private String brandName;
    @Value("${price}")
    private int price;

    Motor motor;

    public void pressStartKey() {
        motor.start();
        System.out.println("Vehicle started");
    }

    public void setBrandName(String brandName) {
        this.brandName = brandName;
    }

    public void setPrice(int price) {
        this.price = price;
    }

    @Autowired
    public void setMotor(Motor motor) {
        this.motor = motor;
    }

    @Override
    public String toString() {
        return "Vehicle [brandName=" + brandName + ", price=" + price + ",
motor=" + motor + "];";
    }
}

```

For all these values, we don't need to write the property source separately, because inside the IoC container an Environment object is created, and the PropertySource resides inside that Environment, containing all the keys and values from the same properties file.

`application.properties`

```
companyName=TATA
capacity=10
brandName=Enfield
price=200000
```

Now let's look at the main method. Since we are using the Java Configuration approach, we have to create the IoC container of type `AnnotationConfigApplicationContext`.

`Main.java`

```
package com.day29;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context = new
            AnnotationConfigApplicationContext(JavaConfig.class);

        Vehicle bean = context.getBean("vehicle", Vehicle.class);
        System.out.println(bean);
    }
}
```

Adding Component Scan to the Same Java Configuration

The Java Configuration class is loaded by the IoC container. Initially, the container reads only the bean definitions that are declared using `@Bean` methods inside the configuration class.

In our example, we created a bean definition for the Motor class because we don't have its source code and therefore cannot annotate it with `@Component`.

For the Vehicle class, we do have the source code, so instead of creating its bean manually with `@Bean`, we can annotate it with `@Component`.

However, for the IoC container to detect and register classes annotated with `@Component`, we must enable component scanning by using `@ComponentScan(basePackages = "package_name")` in the Java Configuration class.

Let's add the component scan in the same Java Configuration approach.

JavaConfig.java

```
package com.day29;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;
import org.springframework.core.env.Environment;

@Configuration
@PropertySource("classpath:com/day29/application.properties")
@ComponentScan(basePackages = "com.day29")
public class JavaConfig {

    @Autowired
    Environment environment;

    @Bean
    public Motor motor() {
        Motor motor = new Motor();

        motor.setCompanyName(environment.getProperty("companyName"));
        motor.setCapacity(
            Integer.parseInt(environment.getProperty("capacity")));
        return motor;
    }
}
```

And let's call it with the same main method.

Main.java

```
package com.day29;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context = new
            AnnotationConfigApplicationContext(JavaConfig.class);

        Vehicle bean = context.getBean("vehicle", Vehicle.class);
        System.out.println(bean);
    }
}
```

OUTPUT

```
Vehicle [brandName=Enfield, price=200000, motor=Motor [companyName=TATA, capacity=10]]
```

Understanding @ComponentScan

What is @ComponentScan?

@ComponentScan is an annotation that tells the Spring IoC container where it should look for classes that are annotated with Spring stereotypes such as @Component, @Service, @Repository, and @Controller.

When the container starts, it does not automatically search the entire project. It only knows about the beans that we have defined. If we want Spring to find annotated classes automatically, we need to tell it which package to scan. This is exactly what @ComponentScan does.

After scanning the specified package, Spring identifies all the eligible classes, creates their objects, and manages them as beans inside the IoC container.

Why do we use @ComponentScan?

We use @ComponentScan to avoid creating every bean manually.

Suppose we have written our own classes, and since we have access to their source code, we annotate them with @Component. Even after adding @Component, Spring will not create an object for those classes unless it knows where those classes are located.

By using @ComponentScan, we simply tell Spring: "Search this package and automatically create beans for all the classes that are annotated with Spring annotations."

This reduces manual configuration and makes the application easier to maintain.

When should we use @ComponentScan?

We use @ComponentScan whenever we want Spring to automatically create beans for the classes whose source code we own.

For example, if we have a class like:

```
@Component
public class Vehicle {

}
```

Adding only @Component is not enough. The IoC container must be instructed to search the package containing this class. Therefore, we use @ComponentScan.

If @ComponentScan is not present, the Vehicle class will not be detected, and no bean will be created for it.

Where do we use @ComponentScan?

@ComponentScan is generally placed on the Java Configuration class, because this class is responsible for providing configuration information to the Spring IoC container.

Example:

```
@Configuration
@ComponentScan(basePackages = "com.day29")
public class JavaConfiguration {

}
```

Here, com.day29 is the package that Spring will scan. Any class inside this package (or its sub-packages) annotated with `@Component`, `@Service`, `@Repository`, or `@Controller` will automatically become a Spring bean.